

- Description

This module manages a smart tire pressure monitoring system, ensuring that:

1. **The tire pressure is continuously measured to ensure optimal performance and safety.**
2. **If the tire pressure falls below the minimum threshold, a low-pressure alert is triggered.**
3. **If the tire pressure exceeds the maximum threshold, a high-pressure alert is issued.**
4. **If the tire pressure remains within the optimal range, the system indicates a normal state.**

This system helps maintain vehicle safety and performance by ensuring tire pressure remains within acceptable limits.

- Specification

module SmartTirePressureMonitoringEx

var currentPressure, minPressure, maxPressure: real

process updatePressure(changeAmount: real)

functional scenario form:

currentPressure < minPressure && output "Low Tire Pressure Alert: " + currentPressure + " psi", Low tire pressure alert is triggered.

||

currentPressure > maxPressure && output "High Tire Pressure Alert: " + currentPressure + " psi", High tire pressure alert is triggered.

||

currentPressure <= maxPressure && output "Tire Pressure is Optimal: " + currentPressure + " psi", Tire pressure is within the optimal range.

end process

end module

- Java Code

```
package code;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.text.DecimalFormat;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;
import java.util.List;
```

```

import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.atomic.AtomicBoolean;
/**
 * =====
 * c132_SmartTireSystemEx
 * Domain: Smart Tire (TPMS-like) with pressure monitoring and safety.
 *
 * Goals:
 * - Keep the original logic in ONE core function:
 *   public synchronized void updatePressure(double changeAmount)
 * Branches under test:
 *   (1) currentPressure += changeAmount;
 *   (2) if (currentPressure < minPressure) -> "Low"
 *       else if (currentPressure > maxPressure) -> "High"
 *       else -> "Optimal"
 *   (3) Print/log statements preserved.
 *
 * Features:
 * - Sensors: pressure (with noise), temperature, leak-
rate, speed estimator, health index
 * - Actuator (Inflator): clamps inflate/deflate rates but DOES NOT de
cide thresholds
 * - Logger + Alerts (file-backed)
 * - Swing GUI: live status, thresholds, inflator limits, simulation c
ontrols,
 *           alerts, and log tail
 * - Simulation engine: ambient temp, road heat, slow leak, speed load
 * - Replay engine: parse "Env step: ..." lines from log to replay
 * - CLI harness: VT/FT/Z3/UVT with "comment 3 lines to run one group"
 *
 * Compile: javac code/c132_SmartTireSystemEx.java
 * Run GUI: java code.c132_SmartTireSystemEx
 * Run CLI: java code.c132_SmartTireSystemEx --cli
 * =====
 */
/* ===== Utilities ===== */
class TStr {
    static String fmt2(double v) { return new DecimalFormat("0.00").for
mat(v); }
    static String fmt3(double v) { return new DecimalFormat("0.000").fo
rmat(v); }
    static String now() { return LocalDateTime.now().format(DateTimeFor
matter.ofPattern("yyyy-MM-dd HH:mm:ss")); }
}

```

```

/* ===== Logger ===== */
class TDataLogger {
    private final File file;
    private final ConcurrentLinkedQueue<String> inMemory = new ConcurrentLinkedQueue<>();
    private final int cap;
    TDataLogger(String path, int inMemoryCap) {
        this.file = new File(path);
        this.cap = Math.max(300, inMemoryCap);
        try {
            if (!file.exists()) {
                File parent = file.getParentFile();
                if (parent != null) parent.mkdirs();
                file.createNewFile();
            }
        } catch (IOException e) {
            System.err.println("[Logger] Create file failed: " + e.getMessage());
        }
    }
    synchronized void log(String message) {
        String line = "[" + TStr.now() + "]" + message;
        inMemory.add(line);
        while (inMemory.size() > cap) inMemory.poll();
        try (FileOutputStream fos = new FileOutputStream(file, true);
            OutputStreamWriter osw = new OutputStreamWriter(fos, StandardCharsets.UTF_8);
            BufferedWriter bw = new BufferedWriter(osw)) {
            bw.write(line);
            bw.newLine();
        } catch (IOException e) {
            System.err.println("[Logger] Write failed: " + e.getMessage());
        }
    }
    List<String> recent(int n) {
        ArrayList<String> list = new ArrayList<>(inMemory);
        if (list.size() <= n) return list;
        return list.subList(list.size() - n, list.size());
    }
    File getFile() { return file; }
}
/* ===== Alerts ===== */
class TAlertSystem {

```

```

enum Severity { INFO, WARNING, CRITICAL }
static class Alert {
    final Severity severity;
    final String message;
    final LocalDateTime time = LocalDateTime.now();
    Alert(Severity s, String m) { this.severity = s; this.message =
m; }
    @Override public String toString() { return "[" + time + "]" +
severity + " " + message; }
}
private final List<Alert> alerts = new ArrayList<>();
private final TDataLogger logger;
TAlertSystem(TDataLogger logger) { this.logger = logger; }
void push(Severity s, String msg) {
    Alert a = new Alert(s, msg);
    alerts.add(a);
    logger.log("ALERT " + s + " :: " + msg);
    System.out.println(a);
}
List<Alert> recent(int n) {
    if (alerts.size() <= n) return new ArrayList<>(alerts);
    return new ArrayList<>(alerts.subList(alerts.size() - n, alerts
.size()));
}
}
/* ===== Sensors ===== */
abstract class TSensor {
    protected String name;
    protected double value;
    protected boolean ok = true;
    protected String status = "OK";
    protected Random rnd = new Random();
    TSensor(String name) { this.name = name; }
    abstract void update(TireContext ctx);
    String getName() { return name; }
    double getValue() { return value; }
    boolean isOk() { return ok; }
    String getStatus() { return status; }
}
class TireContext {
    public double currentPressure; // psi
    public double minPressure;    // psi
    public double maxPressure;    // psi
    public double tireTemperature; // °C

```

```

    public double leakRate;        // psi/s (positive means Leaking)
    public double inflowRate;      // psi/s (from inflator)
    public double speedKmh;        // km/h
    public double healthIndex;     // 0..1 (1 = perfect)
    TireContext copy() {
        TireContext c = new TireContext();
        c.currentPressure = currentPressure;
        c.minPressure = minPressure;
        c.maxPressure = maxPressure;
        c.tireTemperature = tireTemperature;
        c.leakRate = leakRate;
        c.inflowRate = inflowRate;
        c.speedKmh = speedKmh;
        c.healthIndex = healthIndex;
        return c;
    }
}

class PressureSensor132 extends TSensor {
    PressureSensor132() { super("PressureSensor"); }
    @Override
    void update(TireContext ctx) {
        // Add slight noise and drift
        value = Math.max(0, ctx.currentPressure + (rnd.nextDouble() - 0
.5) * 0.2);
        ok = value >= 0 && value < Math.max(120, ctx.maxPressure * 2.0)
;
        status = ok ? (value < ctx.minPressure ? "LOW" : (value > ctx.m
axPressure ? "HIGH" : "OK")) : "BROKEN";
    }
}

class TemperatureSensorT extends TSensor {
    TemperatureSensorT() { super("TemperatureSensor"); }
    @Override
    void update(TireContext ctx) {
        value = ctx.tireTemperature + (rnd.nextDouble() - 0.5) * 0.6;
        ok = value > -40 && value < 150;
        status = ok ? (value > 90 ? "HOT" : "OK") : "OUT_OF_RANGE";
    }
}

class LeakSensor extends TSensor {
    LeakSensor() { super("LeakSensor"); }
    @Override
    void update(TireContext ctx) {
        // Estimate Leak: positive value means losing pressure

```

```

        value = Math.max(0, ctx.leakRate + (rnd.nextDouble() - 0.5) * 0
.01);
        ok = value >= 0 && value < 5;
        status = ok ? (value > 0.2 ? "LEAKY" : "OK") : "BROKEN";
    }
}

class SpeedSensor extends TSensor {
    SpeedSensor() { super("SpeedSensor"); }
    @Override
    void update(TireContext ctx) {
        value = Math.max(0, ctx.speedKmh + (rnd.nextDouble() - 0.5) * 1
.5);
        ok = value >= 0 && value < 400;
        status = ok ? (value > 180 ? "FAST" : "OK") : "BROKEN";
    }
}

class HealthSensor extends TSensor {
    HealthSensor() { super("HealthSensor"); }
    @Override
    void update(TireContext ctx) {
        value = Math.max(0, Math.min(1.0, ctx.healthIndex + (rnd.nextDo
uble() - 0.5) * 0.02));
        ok = value >= 0 && value <= 1.0;
        status = ok ? (value < 0.4 ? "POOR" : (value < 0.7 ? "FAIR" : "
GOOD")) : "BROKEN";
    }
}

/* ===== Inflater (physical helper) =====
===== */
class Inflater {
    private double maxInflateRate = 1.5; // psi/s
    private double maxDeflateRate = 1.5; // psi/s
    double clampInflate(double desiredRate) {
        return Math.max(0.0, Math.min(maxInflateRate, desiredRate));
    }
    double clampDeflate(double desiredRate) {
        return -
Math.max(0.0, Math.min(maxDeflateRate, Math.abs(desiredRate)));
    }
    void setMaxInflate(double r) { maxInflateRate = Math.max(0.2, Math.
min(10.0, r)); }
    void setMaxDeflate(double r) { maxDeflateRate = Math.max(0.2, Math.
min(10.0, r)); }
    double getMaxInflate() { return maxInflateRate; }
}

```

```

    double getMaxDeflate() { return maxDeflateRate; }
}
/* ===== Core Domain (SmartTire) =====
===== */
class SmartTire {
    private final TDataLogger logger;
    private final TAlertSystem alert;
    // thresholds
    private double minPressure; // psi
    private double maxPressure; // psi
    // state
    private double currentPressure; // psi
    // environment/process
    private double tireTemperature = 30.0; // °C
    private double leakRate = 0.02; // psi/s loss baseline
    private double speedKmh = 0.0; // km/h (load effect)
    private double healthIndex = 0.9; // 0..1 (aging, damage)
    // inflator
    private final Inflator inflator = new Inflator();
    private double inflowRate = 0.0; // psi/s (+ means inflating,
- means deflating)
    // sensors
    private final PressureSensor132 pressureSensor = new PressureSensor
132();
    private final TemperatureSensorT tempSensor = new TemperatureSensor
T();
    private final LeakSensor leakSensor = new LeakSensor();
    private final SpeedSensor speedSensor = new SpeedSensor();
    private final HealthSensor healthSensor = new HealthSensor();
    private final List<Runnable> listeners = new ArrayList<>();
    SmartTire(double minPressure, double maxPressure, double initialPre
ssure,
                TDataLogger logger, TAlertSystem alert) {
        this.minPressure = minPressure;
        this.maxPressure = maxPressure;
        this.currentPressure = initialPressure;
        this.logger = logger;
        this.alert = alert;
        logger.log("SmartTire initialized. Pressure=" + TStr.fmt2(curre
ntPressure) + " psi"
                + " (min=" + TStr.fmt2(minPressure) + ", max=" + TStr.f
mt2(maxPressure) + ")");
    }
    void addListener(Runnable r) { listeners.add(r); }

```

```

    private void emit() { for (Runnable r : listeners) try { r.run(); }
catch (Throwable ignored) {} }
    TireContext buildContext() {
        TireContext c = new TireContext();
        c.currentPressure = currentPressure;
        c.minPressure = minPressure;
        c.maxPressure = maxPressure;
        c.tireTemperature = tireTemperature;
        c.leakRate = leakRate;
        c.inflowRate = inflowRate;
        c.speedKmh = speedKmh;
        c.healthIndex = healthIndex;
        return c;
    }
    private void sampleSensors() {
        TireContext ctx = buildContext();
        pressureSensor.update(ctx);
        tempSensor.update(ctx);
        leakSensor.update(ctx);
        speedSensor.update(ctx);
        healthSensor.update(ctx);
        // Alerts based on safe envelopes
        if (currentPressure < minPressure) {
            alert.push(TAlertSystem.Severity.WARNING, "Below MIN pressure (" + TStr.fmt2(currentPressure) + " < " + TStr.fmt2(minPressure) + " psi)");
        } else if (currentPressure > maxPressure) {
            alert.push(TAlertSystem.Severity.WARNING, "Above MAX pressure (" + TStr.fmt2(currentPressure) + " > " + TStr.fmt2(maxPressure) + " psi)");
        }
        if (tempSensor.getValue() > 95) {
            alert.push(TAlertSystem.Severity.CRITICAL, "Tire overheat: " + TStr.fmt2(tempSensor.getValue()) + " °C");
        }
        if (leakSensor.getValue() > 0.5) {
            alert.push(TAlertSystem.Severity.WARNING, "Leak too fast: " + TStr.fmt2(leakSensor.getValue()) + " psi/s");
        }
        if (healthSensor.getValue() < 0.35) {
            alert.push(TAlertSystem.Severity.WARNING, "Tire health poor : idx=" + TStr.fmt2(healthSensor.getValue()));
        }
    }
}

```

```

/* =====
* CORE FUNCTION: Keep the original logic in ONE function.
* Branches to cover:
*   - currentPressure += changeAmount;
*   - if (< min) -> "Low" ; else if (> max) -> "High" ; else -> "Optimal"
*   - Print/log statements preserved.
* Extended tasks (inflator/physics/sensors) happen around it
* but the decision remains exclusively here.
* ===== */
public synchronized void updatePressure(double changeAmount) {
    // (1) Update pressure by external delta (inflate/deflate/Leak/
    // temperature effects)
    currentPressure += changeAmount;
    // if (currentPressure < 0) currentPressure = 0; // physical clamp
    // (2) Print/Log per original format + decision in one place
    String line = "Current Tire Pressure: " + TStr.fmt2(currentPressure) + " psi";
    logger.log(line);
    System.out.println(line);
    if (currentPressure < minPressure) {
        System.out.println("ALERT: Low Tire Pressure!");
    } else if (currentPressure > maxPressure) {
        System.out.println("ALERT: High Tire Pressure!");
    } else {
        System.out.println("Tire Pressure is Optimal.");
    }
    // (3) Sensors/alerts
    sampleSensors();
    emit();
}

/* ===== Simulation hooks ===== */
synchronized void tickSeconds(double seconds) {
    // Leaks reduce pressure; temperature rise from speed increases
    // pressure (~0.01 psi/°C); small passive equalization
    double tempRise = Math.max(0, (0.02 * speedKmh) * seconds); //
    // road heating proxy
    tireTemperature = Math.max(-
30, Math.min(130, tireTemperature + tempRise - 0.03 * seconds));
    // Ideal gas-ish small adjustment:  $\Delta P \approx k * \Delta T$  (toy)
    double dP_temp = (tireTemperature - 25.0) * 0.005 * seconds;
    // Active inflow/deflow (actuator clamps)
    double inflow = 0.0;

```

```

        if (inflowRate > 0) inflow = inflater.clampInflate(inflowRate)
* seconds;
        if (inflowRate < 0) inflow = inflater.clampDeflate(inflowRate)
* seconds; // negative
        // Leak loss
        double leakLoss = -Math.max(0, leakRate) * seconds;
        // Load effect (speed compress/load -> small loss)
        double loadLoss = -Math.max(0, speedKmh) * 0.0005 * seconds;
        // Use the same core function to keep coverage focused there
        updatePressure(dP_temp + inflow + leakLoss + loadLoss);
        // Health degrades slowly at high temp
        healthIndex = Math.max(0, Math.min(1.0, healthIndex - (tireTemp
erature > 80 ? 0.0004 : 0.0001) * seconds));
    }
    synchronized void applyEnvironment(double dSpeed, double dTempAmb,
double dLeak, double inflowSet) {
        speedKmh = Math.max(0, speedKmh + dSpeed);
        tireTemperature += dTempAmb; // ambient influences surface temp
a bit
        leakRate = Math.max(0, leakRate + dLeak);
        inflowRate = inflowSet; // desired inflow setpoint; actuator cl
amps in tick
        logger.log(String.format(Locale.US,
            "Env step: dV=%.2f km/h, dTa=%.2f °C, dLeak=%.3f psi/s,
inflow=%.3f psi/s | V=%.1f, T=%.1f, Leak=%.3f, In=%.3f",
            dSpeed, dTempAmb, dLeak, inflowSet, speedKmh, tireTempe
rature, leakRate, inflowRate));
        emit();
    }
    /* ===== Maintenance/Config ===== */
    synchronized void calibratePressure(double observedPsi) {
        double old = currentPressure;
        currentPressure = Math.max(0, Math.min(200, observedPsi));
        logger.log("Calibration: pressure " + TStr.fmt2(old) + " -> " +
TStr.fmt2(currentPressure) + " psi");
        sampleSensors();
        emit();
    }
    synchronized void setThresholds(double minP, double maxP) {
        double oldMin = minPressure, oldMax = maxPressure;
        minPressure = Math.max(5, Math.min(minP, maxP - 0.5));
        maxPressure = Math.max(minPressure + 0.5, maxP);
        logger.log("Thresholds changed: min " + TStr.fmt2(oldMin) + "->
" + TStr.fmt2(minPressure) +

```

```

        ", max " + TStr.fmt2(oldMax) + "->" + TStr.fmt2(maxPressure));
        sampleSensors();
        emit();
    }
    synchronized void setInflatorLimits(double maxInflate, double maxDeflate) {
        inflator.setMaxInflate(maxInflate);
        inflator.setMaxDeflate(maxDeflate);
        logger.log("Inflator limits set: +max=" + TStr.fmt2(maxInflate)
+ " psi/s, -max=" + TStr.fmt2(maxDeflate) + " psi/s");
    }
    /* ===== Getters for GUI/Tests ===== */
    double getCurrentPressure() { return currentPressure; }
    double getMinPressure() { return minPressure; }
    double getMaxPressure() { return maxPressure; }
    double getTireTemperature() { return tireTemperature; }
    double getLeakRate() { return leakRate; }
    double getSpeedKmh() { return speedKmh; }
    double getHealthIndex() { return healthIndex; }
    double getInflowRate() { return inflowRate; }
    PressureSensor132 getPressureSensor() { return pressureSensor; }
    TemperatureSensorT getTempSensor() { return tempSensor; }
    LeakSensor getLeakSensor() { return leakSensor; }
    SpeedSensor getSpeedSensor() { return speedSensor; }
    HealthSensor getHealthSensor() { return healthSensor; }
}
/* ===== Simulation Engine =====
= */
class TireSimulationEngine implements Runnable {
    private final SmartTire tire;
    private final TDataLogger logger;
    private final AtomicBoolean running = new AtomicBoolean(false);
    private Thread thread;
    private volatile double dSpeed = +1.2;    // km/h per tick
    private volatile double dAmb = +0.05;    // °C per tick to tire surface
    private volatile double dLeak = +0.001;    // psi/s per tick
    private volatile double inflowSet = 0.0;    // psi/s target; negative means deflate
    private volatile double tickSeconds = 1.0;
    private volatile long sleepMillis = 250L;
    private volatile boolean randomize = true;
    private final Random rnd = new Random();

```

```

TireSimulationEngine(SmartTire tire, TDataLogger logger) {
    this.tire = tire;
    this.logger = logger;
}
void start() {
    if (running.get()) return;
    running.set(true);
    thread = new Thread(this, "TireSimEngine");
    thread.setDaemon(true);
    thread.start();
    logger.log("Tire simulation started.");
}
void stop() {
    running.set(false);
    if (thread != null) {
        try { thread.join(2000); } catch (InterruptedException ignored) {}
    }
    logger.log("Tire simulation stopped.");
}
boolean isRunning() { return running.get(); }
void setParams(double dSpeed, double dAmb, double dLeak, double inflowSet,
               double tickSeconds, long delayMs, boolean randomize)
{
    this.dSpeed = dSpeed;
    this.dAmb = dAmb;
    this.dLeak = dLeak;
    this.inflowSet = inflowSet;
    this.tickSeconds = Math.max(0.1, tickSeconds);
    this.sleepMillis = Math.max(20, delayMs);
    this.randomize = randomize;
    logger.log(String.format(Locale.US,
        "Sim params updated: dV=%.3f km/h, dTa=%.3f °C, dLeak=%.4f psi/s, inflow=%.3f psi/s, dt=%.2fs, delay=%dms, random=%s",
        dSpeed, dAmb, dLeak, inflowSet, this.tickSeconds, this.sleepMillis, String.valueOf(this.randomize)));
}
@Override
public void run() {
    while (running.get()) {
        double v = dSpeed, a = dAmb, lk = dLeak, infl = inflowSet;
        if (randomize) {

```

```

        v += (rnd.nextDouble() - 0.5) * Math.max(1.5, Math.abs(
dSpeed) * 2.0);
        a += (rnd.nextDouble() - 0.5) * Math.max(0.2, Math.abs(
dAmb) * 2.0);
        lk = Math.max(0, lk + (rnd.nextDouble() - 0.5) * Math.m
ax(0.01, Math.abs(dLeak)));
        infl += (rnd.nextDouble() - 0.5) * Math.max(0.3, Math.a
bs(inflowSet));
    }
    tire.applyEnvironment(v, a, lk, infl);
    tire.tickSeconds(tickSeconds);
    try { Thread.sleep(sleepMillis); }
    catch (InterruptedException e) { break; }
}
}
}
/* ===== Replay Engine ===== */
class TireReplayEngine {
    private final SmartTire tire;
    private final TDataLogger logger;
    TireReplayEngine(SmartTire tire, TDataLogger logger) {
        this.tire = tire;
        this.logger = logger;
    }
    void replay(File logFile, long delayMs) {
        if (logFile == null || !logFile.exists()) {
            System.err.println("[Replay] Log file not found.");
            return;
        }
        logger.log("Replay started from: " + logFile.getAbsolutePath())
;
        try (BufferedReader br = new BufferedReader(new InputStreamRead
er(new FileInputStream(logFile), StandardCharsets.UTF_8))) {
            String line;
            while ((line = br.readLine()) != null) {
                if (!line.contains("Env step:")) continue;
                try {
                    int idx = line.indexOf("Env step:");
                    String seg = line.substring(idx);
                    // Env step: dV=..., dTa=..., dLeak=..., inflow=...
                    / ...
                    String[] parts = seg.split("\\|")[0].split(",");
                    double v = Double.parseDouble(parts[0].split("=")[1
].replace("km/h", "").trim());

```

```

        double a = Double.parseDouble(parts[1].split("=")[1]
].replace("°C", "").trim());
        double lk = Double.parseDouble(parts[2].split("=")[1]
].replace("psi/s", "").trim());
        double infl = Double.parseDouble(parts[3].split("=")
)[1].replace("psi/s", "").trim());
        tire.applyEnvironment(v, a, lk, infl);
        tire.tickSeconds(1.0);
        Thread.sleep(delayMs);
    } catch (Throwable ignored) {}
    }
} catch (IOException e) {
    System.err.println("[Replay] Error: " + e.getMessage());
}
logger.log("Replay finished.");
}
}
/* ===== GUI ===== */
class TireGUI extends JFrame {
    private final SmartTire tire;
    private final TDataLogger logger;
    private final TAlertSystem alerts;
    private final TireSimulationEngine engine;
    private final TireReplayEngine replay;
    private JLabel lblP, lblMin, lblMax, lblT, lblLeak, lblSpeed, lblHe
alth, lblInflow;
    private JTable tblSensors;
    private DefaultTableModel tblModel;
    private JTextArea txtLog, txtAlerts;
    private JSpinner spMin, spMax, spInfPlus, spInfMinus;
    private JSpinner spdV, spdAmb, spdLeak, spInflow, spDt, spDelay;
    private JCheckBox cbRandomize;
    private JButton btnStart, btnStop, btnReplay, btnCalib, btnInflate,
btnDeflate, btnApplyTh, btnApplyInfl;
    private javax.swing.Timer uiTimer;
    TireGUI(SmartTire tire, TDataLogger logger, TAlertSystem alerts,
        TireSimulationEngine engine, TireReplayEngine replay) {
        super("SmartTire - Pressure Monitoring & Safety");
        this.tire = tire;
        this.logger = logger;
        this.alerts = alerts;
        this.engine = engine;
        this.replay = replay;
        setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);

```

```

        buildUi();
        setSize(1240, 800);
        setLocationRelativeTo(null);
        uiTimer = new javax.swing.Timer(500, e -> refreshUi());
        uiTimer.start();
        tire.addListener(this::refreshUi);
    }
    private JPanel titled(String title, JComponent inner) {
        JPanel p = new JPanel(new BorderLayout());
        p.setBorder(BorderFactory.createTitledBorder(title));
        p.add(inner, BorderLayout.CENTER);
        return p;
    }
    private void buildUi() {
        JPanel root = new JPanel(new BorderLayout(10,10));
        root.setBorder(new EmptyBorder(10,10,10,10));
        setContentPane(root);
        /* LEFT: Status & Controls */
        JPanel left = new JPanel();
        left.setLayout(new BoxLayout(left, BoxLayout.Y_AXIS));
        JPanel status = new JPanel(new GridLayout(0,2,6,6));
        lblP = new JLabel("-");
        lblMin = new JLabel("-");
        lblMax = new JLabel("-");
        lblT = new JLabel("-");
        lblLeak = new JLabel("-");
        lblSpeed = new JLabel("-");
        lblHealth = new JLabel("-");
        lblInflow = new JLabel("-");
        status.add(new JLabel("Pressure (psi):"));    status.add(lblP);
        status.add(new JLabel("Min (psi):"));        status.add(lblMin);
        status.add(new JLabel("Max (psi):"));        status.add(lblMax);
        status.add(new JLabel("Temperature (°C):")); status.add(lblT);
        status.add(new JLabel("Leak (psi/s):"));     status.add(lblLeak);
;
        status.add(new JLabel("Speed (km/h):"));     status.add(lblSpeed);
);
        status.add(new JLabel("Health:"));           status.add(lblHealth);
h);
        status.add(new JLabel("Inflow (psi/s):"));   status.add(lblInflow);
w);
        left.add(titled("Live Status", status));
        JPanel th = new JPanel(new GridLayout(0,2,6,6));

```

```

spMin = new JSpinner(new SpinnerNumberModel(30.0, 5.0, 80.0, 0.5));
spMax = new JSpinner(new SpinnerNumberModel(35.0, 10.0, 100.0, 0.5));
btnApplyTh = new JButton("Apply Thresholds");
btnApplyTh.addActionListener(e -> {
    double mn = ((Number)spMin.getValue()).doubleValue();
    double mx = ((Number)spMax.getValue()).doubleValue();
    tire.setThresholds(mn, mx);
});
th.add(new JLabel("Min pressure (psi)")); th.add(spMin);
th.add(new JLabel("Max pressure (psi)")); th.add(spMax);
th.add(new JLabel(" ")); th.add(btnApplyTh);
left.add(Box.createVerticalStrut(6));
left.add(titled("Thresholds", th));
JPanel infl = new JPanel(new GridLayout(0,2,6,6));
spInfPlus = new JSpinner(new SpinnerNumberModel(1.5, 0.2, 10.0, 0.1));
spInfMinus= new JSpinner(new SpinnerNumberModel(1.5, 0.2, 10.0, 0.1));
btnApplyInfl = new JButton("Apply Inflator Limits");
btnApplyInfl.addActionListener(e -> tire.setInflatorLimits(
    ((Number)spInfPlus.getValue()).doubleValue(),
    ((Number)spInfMinus.getValue()).doubleValue()
));
infl.add(new JLabel("Max Inflate (psi/s)")); infl.add(spInfPlus);
infl.add(new JLabel("Max Deflate (psi/s)")); infl.add(spInfMinus);
infl.add(new JLabel(" ")); infl.add(btnApplyInfl);
left.add(Box.createVerticalStrut(6));
left.add(titled("Inflator", infl));
JPanel ops = new JPanel(new GridLayout(0,2,6,6));
btnInflate = new JButton("Manual Inflate +2 psi");
btnDeflate = new JButton("Manual Deflate -2 psi");
btnCalib = new JButton("Calibrate Pressure");
btnInflate.addActionListener(e -> tire.updatePressure(+2.0));
btnDeflate.addActionListener(e -> tire.updatePressure(-2.0));
btnCalib.addActionListener(e -> {
    String s = JOptionPane.showInputDialog(this, "Observed pressure (psi):", TStr.fmt2(tire.getCurrentPressure()));
    if (s == null) return;
    try { tire.calibratePressure(Double.parseDouble(s)); }

```

```

        catch (NumberFormatException ex) { JOptionPane.showMessageDialog(
ialog(this, "Invalid number."); }
    });
    ops.add(btnInflate); ops.add(btnDeflate);
    ops.add(btnCalib); ops.add(new JLabel(" "));
    left.add(Box.createVerticalStrut(6));
    left.add(titled("Operations", ops));
    JPanel sim = new JPanel(new GridLayout(0,2,6,6));
    spdV = new JSpinner(new SpinnerNumberModel(+1.2, -
20.0, 30.0, 0.5));
    spdAmb = new JSpinner(new SpinnerNumberModel(+0.05, -
5.0, 5.0, 0.05));
    spdLeak= new JSpinner(new SpinnerNumberModel(+0.001, 0.0, 0.1,
0.001));
    spInflow= new JSpinner(new SpinnerNumberModel(0.0, -
5.0, 5.0, 0.1));
    spDt = new JSpinner(new SpinnerNumberModel(1.0, 0.1, 10.0, 0.1
));
    spDelay= new JSpinner(new SpinnerNumberModel(250, 20, 2000, 10)
);

    cbRandomize = new JCheckBox("Randomize", true);
    btnStart = new JButton("Start Simulation");
    btnStop = new JButton("Stop Simulation");
    btnReplay= new JButton("Replay from Log");
    btnStart.addActionListener(e -> engine.setParams(
        ((Number)spdV.getValue()).doubleValue(),
        ((Number)spdAmb.getValue()).doubleValue(),
        ((Number)spdLeak.getValue()).doubleValue(),
        ((Number)spInflow.getValue()).doubleValue(),
        ((Number)spDt.getValue()).doubleValue(),
        ((Number)spDelay.getValue()).longValue(),
        cbRandomize.isSelected()
    ));
    btnStart.addActionListener(e -> engine.start());
    btnStop.addActionListener(e -> engine.stop());
    btnReplay.addActionListener(e -> new Thread(() -> replay.replay
(logger.getFile(), 60), "Replay").start());
    sim.add(new JLabel("ΔSpeed (km/h/tick)")); sim.add(spdV);
    sim.add(new JLabel("ΔAmbient (°C/tick)")); sim.add(spdAmb);
    sim.add(new JLabel("ΔLeak (psi/s/tick)")); sim.add(spdLeak);
    sim.add(new JLabel("Inflow set (psi/s)")); sim.add(spInflow);
    sim.add(new JLabel("dt (s)")); sim.add(spDt);
    sim.add(new JLabel("Delay (ms)")); sim.add(spDelay);
    sim.add(new JLabel(" ")); sim.add(cbRandomize);

```

```

        sim.add(btnStart);
        sim.add(new JLabel(" "));
        left.add(Box.createVerticalStrut(6));
        left.add(titled("Simulation", sim));
        getContentPane().add(left, BorderLayout.WEST);
        /* CENTER: Sensors + Alerts */
        JPanel center = new JPanel(new BorderLayout(6,6));
        tblModel = new DefaultTableModel(new String[]{"Sensor", "Value",
"OK", "Status"}, 0) {
            @Override public boolean isCellEditable(int row, int column
) { return false; }
        };
        tblSensors = new JTable(tblModel);
        JScrollPane spTbl = new JScrollPane(tblSensors);
        center.add(titled("Sensors", spTbl), BorderLayout.CENTER);
        txtAlerts = new JTextArea(8, 60);
        txtAlerts.setEditable(false);
        center.add(titled("Recent Alerts", new JScrollPane(txtAlerts)),
BorderLayout.SOUTH);
        getContentPane().add(center, BorderLayout.CENTER);
        /* RIGHT: Log tail */
        txtLog = new JTextArea();
        txtLog.setEditable(false);
        JScrollPane spLog = new JScrollPane(txtLog);
        getContentPane().add(titled("Log (tail)", spLog), BorderLayout.
EAST);
    }
    private void refreshUi() {
        lblP.setText(TStr.fmt2(tire.getCurrentPressure()));
        lblMin.setText(TStr.fmt2(tire.getMinPressure()));
        lblMax.setText(TStr.fmt2(tire.getMaxPressure()));
        lblT.setText(TStr.fmt2(tire.getTireTemperature()));
        lblLeak.setText(TStr.fmt2(tire.getLeakRate()));
        lblSpeed.setText(TStr.fmt2(tire.getSpeedKmh()));
        lblHealth.setText(TStr.fmt2(tire.getHealthIndex()));
        lblInflow.setText(TStr.fmt2(tire.getInflowRate()));
        tblModel.setRowCount(0);
        tblModel.addRow(new Object[]{
            tire.getPressureSensor().getName(),
            TStr.fmt2(tire.getPressureSensor().getValue()),
            tire.getPressureSensor().isOk(), tire.getPressureSensor
().getStatus()
        });
        tblModel.addRow(new Object[]{

```

```

        tire.getTempSensor().getName(),
        TStr.fmt2(tire.getTempSensor().getValue()),
        tire.getTempSensor().isOk(), tire.getTempSensor().getSt
atus()
    });
    tblModel.addRow(new Object[]{
        tire.getLeakSensor().getName(),
        TStr.fmt2(tire.getLeakSensor().getValue()),
        tire.getLeakSensor().isOk(), tire.getLeakSensor().getSt
atus()
    });
    tblModel.addRow(new Object[]{
        tire.getSpeedSensor().getName(),
        TStr.fmt2(tire.getSpeedSensor().getValue()),
        tire.getSpeedSensor().isOk(), tire.getSpeedSensor().get
Status()
    });
    tblModel.addRow(new Object[]{
        tire.getHealthSensor().getName(),
        TStr.fmt2(tire.getHealthSensor().getValue()),
        tire.getHealthSensor().isOk(), tire.getHealthSensor().g
etStatus()
    });
    // alerts
    if (txtAlerts != null) {
        StringBuilder sb = new StringBuilder();
        for (TAlertSystem.Alert a : alerts.recent(18)) sb.append(a)
.append("\n");
        txtAlerts.setText(sb.toString());
        txtAlerts.setCaretPosition(txtAlerts.getDocument().getLengt
h());
    }
    // log tail
    if (txtLog != null) {
        List<String> lines = logger.recent(240);
        StringBuilder sb = new StringBuilder();
        for (String s : lines) sb.append(s).append("\n");
        txtLog.setText(sb.toString());
        txtLog.setCaretPosition(txtLog.getDocument().getLength());
    }
}

/* ===== CLI Harness ===== */
class TireCLIHarness {

```

```

private final SmartTire tire;
TireCLIHarness(SmartTire tire) { this.tire = tire; }
void runAll() {
    System.out.println("==== CLI Demo (VT) =====");
    vt();
    System.out.println("==== CLI Demo (FT) =====");
    ft();
    System.out.println("==== CLI Demo (Z3 mimic) =====");
    z3();
    System.out.println("==== CLI Demo (UVT) =====");
    uvt();
}
// VT: original vector (<min, >max, back to optimal)
void vt() {
    tire.updatePressure(-5.0); // 33 -> 28 (<30) -> Low
    tire.updatePressure(8.0); // 28 -> 36 (>35) -> High
    tire.updatePressure(-2.0); // 36 -> 34 (between) -> Optimal
}
// FT: extreme numeric stress (as user's FT)
void ft() {
    tire.updatePressure(8.812221249325049E307);
    tire.updatePressure(7.986142550757732E295);
    tire.updatePressure(3.4300223128453513E305);
}
// Z3-like: mirror VT
void z3() {
    tire.updatePressure(-1);
    tire.updatePressure(2);
    tire.updatePressure(0);
}
// UVT: mirror VT again
void uvt() {
    tire.updatePressure(-5);
    tire.updatePressure(8);
    tire.updatePressure(0);
}
}
/* ===== Main ===== */
public class c132_SmartTireSystemEx {
    public static void main(String[] args) {
        // boolean forceCli = Arrays.asList(args).contains("--cli");
        boolean forceCli = true;

        // Logger & Alerts

```

```

        TDataLogger logger = new TDataLogger("tire_log.txt", 900);
        TAlertSystem alertSystem = new TAlertSystem(logger);
        // Tire with user's parameters
        SmartTire tire = new SmartTire(30.0, 35.0, 33.0, logger, alertSystem);

        // Engines
        TireSimulationEngine engine = new TireSimulationEngine(tire, logger);
        TireReplayEngine replay = new TireReplayEngine(tire, logger);
        // ===== CLI path (sequential 4 groups; comment 3 lines to run single group) =====
        if (forceCli || headless()) {
            logger.log("Running in CLI mode.");
            TireCLIHarness harness = new TireCLIHarness(tire);
            harness.vt(); // ← keep to run VT
            // harness.ft(); // ← comment out to skip FT
            // harness.z3(); // ← comment out to skip Z3
            // harness.uvt(); // ← comment out to skip UVT
            // small sim to exercise sensors & dynamics
            for (int i = 0; i < 20; i++) {
                tire.applyEnvironment(+1.2, +0.05, +0.001, 0.0);
                tire.tickSeconds(1.0);
                try { Thread.sleep(100L); } catch (Exception ignored) {}
            }
            logger.log("CLI demo finished.");
            System.out.println("CLI demo finished. Log written to: " + logger.getFile().getAbsolutePath());
            return;
        }
        // ===== GUI path =====
        logger.log("Running in GUI mode.");
        SwingUtilities.invokeLater(() -> {
            TireGUI gui = new TireGUI(tire, logger, alertSystem, engine, replay);
            gui.setVisible(true);
        });
    }
    private static boolean headless() {
        try { return GraphicsEnvironment.isHeadless(); }
        catch (Throwable t) { return true; }
    }
}

```

- Coverage report

1. Test case number: 3
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c132_SmartTireSystemEx.java	40.7 %	44	64	108
> TAlertSystem	0.0 %	0	2	2
> TireReplayEngine	0.0 %	0	8	8
> TireSimulationEngine	0.0 %	0	8	8
> TDataLogger	25.0 %	2	6	8
> c132_SmartTireSystemEx	50.0 %	3	3	6
> HealthSensor	50.0 %	5	5	10
> LeakSensor	50.0 %	4	4	8
> SpeedSensor	50.0 %	4	4	8
> TemperatureSensorT	50.0 %	4	4	8
▼ SmartTire	68.2 %	15	7	22
emit()	50.0 %	1	1	2
tickSeconds(double)	50.0 %	3	3	6
sampleSensors()	70.0 %	7	3	10
updatePressure(double)	100.0 %	4	0	4
SmartTire(double, double, double, TI		0	0	0
applyEnvironment(double, double, d		0	0	0
buildContext()		0	0	0

3. Vibration testing instruction coverage: 100%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c132_SmartTireSystemEx.java	31.9 %	1,010	2,160	3,170
> TireReplayEngine	6.2 %	9	136	145
> TireContext	7.3 %	3	38	41
> TireSimulationEngine	15.5 %	41	223	264
> Inflator	19.1 %	9	38	47
> TireCLIHarness	24.1 %	19	60	79
> TStr	54.5 %	12	10	22
> TAlertSystem	55.9 %	33	26	59
> TDataLogger	57.9 %	70	51	121
> c132_SmartTireSystemEx	68.2 %	75	35	110
> TSensor	69.0 %	20	9	29
▼ SmartTire	69.7 %	417	181	598
emit()	52.9 %	9	8	17
sampleSensors()	74.0 %	77	27	104
tickSeconds(double)	83.2 %	89	18	107
SmartTire(double, double, double, TI	100.0 %	78	0	78
applyEnvironment(double, double, d	100.0 %	80	0	80
buildContext()	100.0 %	38	0	38
updatePressure(double)	100.0 %	46	0	46
> HealthSensor	89.7 %	52	6	58

4. Fuzz testing branch coverage: 50%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c132_SmartTireSystemEx.java	33.3 %	36	72	108
> TAlertSystem	0.0 %	0	2	2
> TireReplayEngine	0.0 %	0	8	8
> TireSimulationEngine	0.0 %	0	8	8
> TDataLogger	25.0 %	2	6	8
> PressureSensor132	30.0 %	3	7	10
> c132_SmartTireSystemEx	50.0 %	3	3	6
> HealthSensor	50.0 %	5	5	10
> LeakSensor	50.0 %	4	4	8
▼ SmartTire	50.0 %	11	11	22
emit()	50.0 %	1	1	2
sampleSensors()	50.0 %	5	5	10
tickSeconds(double)	50.0 %	3	3	6
updatePressure(double)	50.0 %	2	2	4
SmartTire(double, double, double, TI		0	0	0
applyEnvironment(double, double, d		0	0	0
buildContext()		0	0	0
> SpeedSensor	50.0 %	4	4	8
> TemperatureSensorT	50.0 %	4	4	8
> Inflator		0	0	0

5. Fuzz testing instruction coverage: 84.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c132_SmartTireSystemEx.java	30.7 %	973	2,197	3,170
> TireReplayEngine	6.2 %	9	136	145
> TireContext	7.3 %	3	38	41
> TireSimulationEngine	15.5 %	41	223	264
> Inflator	19.1 %	9	38	47
> TireCLIHarness	24.1 %	19	60	79
> TStr	54.5 %	12	10	22
> TAlertSystem	55.9 %	33	26	59
> TDataLogger	57.9 %	70	51	121
▼ SmartTire	66.6 %	398	200	598
▪ emit()	52.9 %	9	8	17
▪ sampleSensors()	62.5 %	65	39	104
▲ tickSeconds(double)	83.2 %	89	18	107
● updatePressure(double)	84.8 %	39	7	46
▲ SmartTire(double, double, double, TI	100.0 %	78	0	78
▲ applyEnvironment(double, double, d	100.0 %	80	0	80
▲ buildContext()	100.0 %	38	0	38
> c132_SmartTireSystemEx	68.2 %	75	35	110
> PressureSensor132	68.3 %	43	20	63
> TSensor	69.0 %	20	9	29

6. Z3 branch coverage: 50%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c132_SmartTireSystemEx.java	35.2 %	38	70	108
> TAlertSystem	0.0 %	0	2	2
> TireReplayEngine	0.0 %	0	8	8
> TireSimulationEngine	0.0 %	0	8	8
> TDataLogger	25.0 %	2	6	8
> c132_SmartTireSystemEx	50.0 %	3	3	6
> HealthSensor	50.0 %	5	5	10
> LeakSensor	50.0 %	4	4	8
> PressureSensor132	50.0 %	5	5	10
▼ SmartTire	50.0 %	11	11	22
▪ emit()	50.0 %	1	1	2
▪ sampleSensors()	50.0 %	5	5	10
▲ tickSeconds(double)	50.0 %	3	3	6
● updatePressure(double)	50.0 %	2	2	4
▲ SmartTire(double, double, double, TI		0	0	0
▲ applyEnvironment(double, double, d		0	0	0
▲ buildContext()		0	0	0

7. Z3 instruction coverage: 82.6 %

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c132_SmartTireSystemEx.java	28.3 %	897	2,273	3,170
> TireReplayEngine	6.2 %	9	136	145
> TireContext	7.3 %	3	38	41
> TireSimulationEngine	15.5 %	41	223	264
> TAlertSystem	18.6 %	11	48	59
> Inflator	19.1 %	9	38	47
> TireCLIHarness	24.1 %	19	60	79
> TStr	54.5 %	12	10	22
> TDataLogger	57.9 %	70	51	121
▼ SmartTire	64.5 %	386	212	598
▪ sampleSensors()	51.9 %	54	50	104
▪ emit()	52.9 %	9	8	17
● updatePressure(double)	82.6 %	38	8	46
▲ tickSeconds(double)	83.2 %	89	18	107
▲ SmartTire(double, double, double, TI	100.0 %	78	0	78
▲ applyEnvironment(double, double, d	100.0 %	80	0	80
▲ buildContext()	100.0 %	38	0	38

8. Unimproved V-method branch coverage: 75%

Element		Coverage	Covered Branches	Missed Branches	Total Branches
▼ c132_SmartTireSystemEx.java		38.0 %	41	67	108
> TAlertSystem		0.0 %	0	2	2
> TireReplayEngine		0.0 %	0	8	8
> TireSimulationEngine		0.0 %	0	8	8
> TDataLogger		25.0 %	2	6	8
> c132_SmartTireSystemEx		50.0 %	3	3	6
> HealthSensor		50.0 %	5	5	10
> LeakSensor		50.0 %	4	4	8
> SpeedSensor		50.0 %	4	4	8
> TemperatureSensorT		50.0 %	4	4	8
▼ SmartTire		59.1 %	13	9	22
emit()		50.0 %	1	1	2
tickSeconds(double)		50.0 %	3	3	6
sampleSensors()		60.0 %	6	4	10
updatePressure(double)		75.0 %	3	1	4
SmartTire(double, double, double, TI			0	0	0
applyEnvironment(double, double, d			0	0	0
buildContext()			0	0	0

9. Unimproved V-method instruction coverage: 93.5%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c132_SmartTireSystemEx.java		31.7 %	1,005	2,165	3,170
> TireReplayEngine		6.2 %	9	136	145
> TireContext		7.3 %	3	38	41
> TireSimulationEngine		15.5 %	41	223	264
> Inflator		19.1 %	9	38	47
> TireCLIHarness		24.1 %	19	60	79
> TStr		54.5 %	12	10	22
> TAlertSystem		55.9 %	33	26	59
> TDataLogger		57.9 %	70	51	121
> c132_SmartTireSystemEx		68.2 %	75	35	110
> TSensor		69.0 %	20	9	29
▼ SmartTire		69.2 %	414	184	598
emit()		52.9 %	9	8	17
sampleSensors()		74.0 %	77	27	104
tickSeconds(double)		83.2 %	89	18	107
updatePressure(double)		93.5 %	43	3	46
SmartTire(double, double, double, TI		100.0 %	78	0	78
applyEnvironment(double, double, d		100.0 %	80	0	80
buildContext()		100.0 %	38	0	38
> HealthSensor		89.7 %	52	6	58